BombLab 实验报告

一．实验过程与核心分析

Phase 1：函数调用与参数传递

本关调用 p1_compute 计算预期值，再与输入比较

assembly

4020db: call 402050 <p1_compute>

4020e0: cmp %eax, -0x14(%rbp)

进入 p1_compute 后发现，它把 64 位的 challenge 拆成高 32 位、中间 16 位和低 32 位，分别送入 p1_step0、p1_step1、p1_step2 计算。

全程按 32 位无符号数处理，公式如下：

- high32 = (uint32_t)(challenge >> 32)
- mid16 = (uint32_t)(challenge >> 16)
- low32 = (uint32_t)challenge
- step1 = (high32 + 4 * mid16) ^ 0x15c8
- step0 = (5 * low32 + high32) ^ 0x55a6
- result = (5 * step0) ^ (step1 + 0xa639)

分析汇编中的移位和异或指令，逆向出了 `challenge` 到目标整数的数学变换公式。

Phase 2 数组、循环与递推

本关需输入 6 个整数，存放在栈上的数组中（基址 -0x40(%rbp)）。前半段验证第 1 个数，后半段通过循环验证第 2~6 个数。

第 1 个数的生成逻辑：

```
402160: shr     $0x4,%rax          # challenge >> 4
402164: add     %eax,%eax          # * 2
402166: and     $0x7e,%eax         # & 0x7e
402169: add     $0x19,%eax         # + 0x19
```

循环递推逻辑（%rdx 是循环变量 i，从 1 到 5）：

```
402180: shr     %cl,%rax           # 按 %cl 右移 challenge
402183: and     $0x1f,%eax         # & 0x1f (取低 5 位)
402186: add     %edx,%eax          # + i
```

识别出典型的数组遍历和循环累加结构，提取出递推关系，利用循环逆向计算出满足条件的初始值。

Phase 3：乱序跳转表分析

汇编通过 `jmp *0x405820(,%rax,8)` 间接跳转。通过 GDB 提取内存发现，跳转表是 乱序映射的 （如索引 0 跳往 case 2）。同时推导出参数 `x = (challenge >> 32) ^ (challenge >> 3)`。

求解逻辑：在 `defuser.c` 中精准还原了乱序的 0~7 索引与计算函数的映射关系，并使用 `switch` 实现了计算公式。

Phase 4：递归调用与返回

关键汇编：入口确定参数 `val = challenge & 0x7f`，核心函数 `p4_walk` 是二分查找的变种。

递归回溯时的计算关系为 `(3 * ret) ^ (mid + mask) ^ 0x1e`。

求解逻辑：在 C 语言中直接复刻了 `p4_walk` 的递归逻辑，省去了手算的麻烦。

Phase 5：结构体、指针与链表

关键汇编：根据 `challenge` 确定链表头和循环次数，遍历计算 `ans = (ans * 3 + node->val1) ^ node->val2`。

易错点：全局数组 `0x405b60` 中存储的头指针是乱序的。通过 GDB 提取地址，构建了真实的 `head_idx` 映射，解决了数据取错导致爆炸的问题。

Phase 6：函数指针与间接调用

关键汇编：两次调用 `call *0x405e00`，但索引不同。第一次为 `idx1 = challenge & 7`，第二次为 `idx2 = (challenge >> 3) & 7`。

求解逻辑：提取了函数指针表，并用 C 语言还原了这 8 个小型计算函数（包含循环左移、循环右移等）。将两次计算结果组合 `ans2 = p6_op(idx2, ans1 ^ 0x4db4)` 得到最终值。

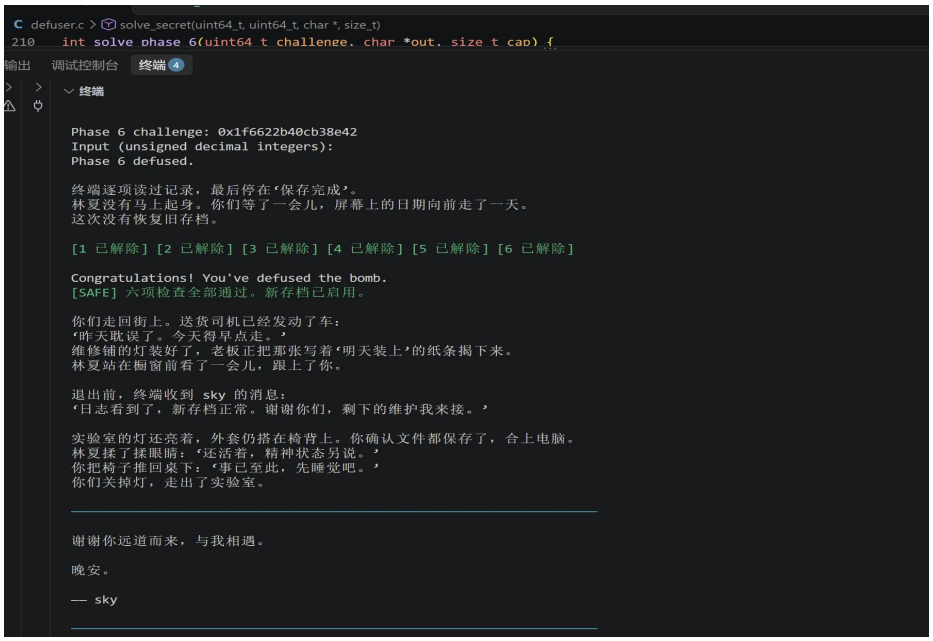
Honor ：栈布局与相邻字段覆盖

关键汇编：`secret_gate` 要求检查偏移 32 字节处的内容为 `"OPEN"`。通过分析栈帧布局，必须在输入字符串前填充 32 个字节，才能让第 33~36 字节正好落在 `tag` 的校验地址上。

计算逻辑：逐行拆解 `secret_phase` 和 `secret_mix` 的寄存器依赖，恢复了 `v1, v2, v3` 的复杂异或与移位公式。

求解逻辑：在输出时使用 `snprintf(out, cap, "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXOPEN %u %u", ...)`，用 32 个字符填充前段，成功绕过长度检查和栈溢出校验。

二．验证记录与截图



```
liuji@localhost:/mnt/c/Users/liuji/ics-2026-bomblab$ python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
```

```
终端

Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
liuji@localhost:/mnt/c/Users/liuji/ics-2026-bomblab$
```

非公开 Seed 验证

```
终端

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。

  这里的东西我会留着。你在北方有了新想法，也可以回来添。
  我暂时不走，这边的事刚有起色，想再试一阵。
  以后说不定会改主意，到时候告诉你。你也一样。
  别因为申请时说得很坚定，遇到不喜欢的事就不好意思讲。

  第一场雪记得拍。我想看看你一直说的北方。

  —— 许遥

后面是这些年断断续续的更新。
```



与辅助工具说明

使用 cladue code 辅导